Axona

Every Public Symbol, Exactly

David A. Smith

Axona.net

The Axona API Reference v4.22.0 2026-07-14

Axona API Reference

Reference for every public symbol exported from `@axona/protocol` v4.22.0.

Organized by what application developers reach for first (identity, peer lifecycle, pub/sub, direct messaging, introspection), then the transport/protocol surface, then low-level utilities. Every signature below is verified against the v4.22.0 kernel source.

> **Which network?** Both live networks run the 4.x line (wire 4.0). **Testnet** (`wss://testnet.axona.net`) tracks the newest kernel — v4.22.0, the version this reference describes. **Production** (`wss://bridge.axona.net` east + `wss://bridge-west.axona.net` west) runs the most recently promoted kernel, typically one release behind while changes soak. The API surface below is identical on both; point wherever you deploy. Install `github:axona-net/axona-protocol#v4.22.0`.

Companion documents:

- Quick Start — 5-minute working roundtrip.
- Programmer Guide — mental model + worked example + pitfalls.
- Security changelog — what each kernel version protects.

Imports throughout assume:

```
import { /* … */ } from '@axona/protocol';
```

Sub-path imports (e.g. `@axona/protocol/contracts/Transport.js`, `@axona/protocol/persistence/file.js`) work for symbols that need them, but most apps only need the main barrel.

> **How to read this reference.** It has three parts. The **Application surface** (§§1–9) is what applications call — most readers never need anything else. The **Transport + protocol surface** (§§10–16) and **Low-level utilities** (§§17–23) exist for transport authors, tooling, and the curious. Building with an AI assistant? Hand it the AI Grounding file — the application surface distilled to rules and patterns.

Table of contents

Application surface (what you'll use first)

6. Mesh introspection + events
7. Envelopes + verification
8. Errors
9. Persistence

Transport + protocol surface

> **You probably don't need this part.** Everything an application calls is in §§1–9 above. The sections from here on are for people implementing transports, embedding the kernel, or building tooling.

10. Contracts
11. Sim transport
12. Web transport
13. Handshake
14. Authenticated-identity handshake (axona/5)
15. Bridge directory
16. Low-level pub/sub builders

Low-level utilities

> **Internals.** Pure helpers the kernel itself is built from. Apps import these only for unusual jobs (custom signing, ID math, region tables).

17. Ed25519 helpers
18. Hex / ID math
19. S2 geographic cells
20. Region names
21. Geo helpers
22. Proof-of-work scaffolding
23. Constants

Application surface

1. Types referenced throughout

`Identity` (node identity)

Returned by `createNodeIdentity`. The connection/transport keypair; its pubkey forms the nodeId.

```
{
  id:         string;       // 66-char lowercase hex (264-bit nodeId)
  pubkey:     Uint8Array;   // 32-byte Ed25519 public key
  pubkeyHex:  string;       // 64-char lowercase hex
  privateKey: CryptoKey;    // Web Crypto Ed25519 (browser + Node 20+)
  region:     { lat: number, lng: number };
```

```
  createdAt:  number;       // ms epoch
  pow:        string;       // transport-role PoW nonce ('' = inert at difficulty 0)
  sign(message: Uint8Array): Promise<Uint8Array>;
  verify(message: Uint8Array, signature: Uint8Array): Promise<boolean>;
}
```

The top byte of `id` is the S2 region cell for `(lat, lng)`; the bottom 256 bits are `SHA-256(pubkey)`. The id is self-authenticating — a peer can only claim an id it holds the private key for.

`AuthorIdentity` (authorship identity)

Returned by `createAuthorIdentity`. A keypair only — **no nodeId, no region** (authorship is not a place). Its public key is the Author ID.

```
{
  kind:       'author';
  authorId:   string;       // 64-char hex Author ID (== signerPubkey on the wire)
  pubkey:     Uint8Array;   // 32-byte Ed25519 public key
  pubkeyHex:  string;       // 64-char hex (same value as authorId)
  privateKey: CryptoKey;
  createdAt:  number;
  pow:        string;       // publish-role PoW nonce ('' = inert)
  sign(message: Uint8Array): Promise<Uint8Array>;
  verify(message: Uint8Array, signature: Uint8Array): Promise<boolean>;
}
```

You pass an `AuthorIdentity` (or `ANONYMOUS`) as `{ signWith }` to `peer.pub` / `kill`.

`TopicDescriptor` (type)

```
{
  region?: string | number;   // region name ('useast'), '0x89', '137', or 137; omit → node
↪  region
  owner?:  string | null;     // 64-hex Author ID, or absent for an open topic
  name:    string;            // required, non-empty
  write?:  'open' | 'owner';  // defaults by owner presence (see §4.1)
}
```

`Envelope` (type)

What `peer.sub` delivers and `peer.pull` returns.

```
{
  msgId:        string;   // 64-char hex = sha256(canonical({ publisher, message }))
  seq:          number;   // per-publisher monotonic sequence (folded under the signature)
  ts:           number;   // ms epoch when published
  topic:        TopicDescriptor;   // the SIGNED descriptor { region, owner, name, write }
```

```
  message:      any;      // your JSON-serializable payload
  signature?:   string;   // 'ed25519:<128 hex>'; present iff signed
  signerPubkey?: string;  // 64-char hex Author ID; present iff signed
  signerPow?:   string;   // publish-role PoW nonce; present iff signed ('' at difficulty 0)
}
```

A **retraction** (see `peer.kill`) is delivered to your `sub` handler as a marker instead of a full envelope:

```
{ deleted: true, msgId: string, topic: string | null }
```

Branch on `env.deleted === true` to drop your local copy of `msgId`.

`Subscription` (type)

Opaque handle returned by `peer.sub()`.

```
sub.id          // string — unique per subscription
sub.topicId     // string — 66-char hex topic ID
sub.topicName   // string — your original topic name (or '#<id-prefix>' for an id-only sub)
sub.stop()      // Promise<void> — cancel; idempotent
```

`PeerId` / `TopicId` (types)

In the public API, peer IDs and topic IDs are **66-char lowercase hex strings**. The kernel holds them as `bigint` internally; conversion happens at the API boundary. Pass strings unless a signature explicitly says BigInt (`peer.lookup`).

---

2. Identity

Two factories. `createNodeIdentity` is the connection key (location + nodeId, used for the handshake and routing). `createAuthorIdentity` is the publish key (no location, used to sign messages). They are separate on purpose (key separation): a peer holds a node identity and signs each publish with an author identity supplied per call.

`createNodeIdentity({ lat, lng, extractable? }) → Promise<Identity>`

Generate a fresh node identity: a new Ed25519 keypair plus the 264-bit nodeId derived from the public key + region.

Param	Type	Notes
`lat`	`number`	required — latitude of the node's region
`lng`	`number`	required — longitude

Param	Type	Notes
`extractable`	`boolean`	default `true`. Pass `false` for an ephemeral browser identity so the signing key can't be exported (XSS can't exfiltrate it). Leave `true` if you will `dumpIdentity` it.

Returns an `Identity`. The top byte of `.id` is the S2 cell for `(lat, lng)`.

Throws `IdentityError`:

- `IDENTITY_INVALID_FORMAT` — `lat`/`lng` not numbers.
- `IDENTITY_KEYGEN_FAILED` — Web Crypto Ed25519 `generateKey` failed.

```
import { createNodeIdentity } from '@axona/protocol';

const node = await createNodeIdentity({ lat: 38.0, lng: -77.0 });
console.log(node.id);             // 'df…'  (66 hex chars)
console.log(node.id.slice(0, 2)); // region byte
```

`createAuthorIdentity({ persistAs?, store?, extractable? }) → Promise<AuthorIdentity>`

Mint (or load-or-create) the authorship key whose public key is the Author ID. Durability is the only real choice: ephemeral (unlinkable) or persisted (a recognizable author across sessions, able to retract).

Param	Type	Notes
`persistAs`	`string`	storage key; load-or-create then save. Omit for an ephemeral author.
`store`	`{ get, set }`	custom store; defaults to browser `localStorage` when `persistAs` is set.
`extractable`	`boolean`	default `true`; **forced `true`** when `persistAs` is set (a persisted key must be exportable).

Returns an `AuthorIdentity`.

Throws `IdentityError` (`IDENTITY_KEYGEN_FAILED`, `IDENTITY_LOAD_FAILED`, `IDENTITY_INVALID_FORMAT`).

```
import { createAuthorIdentity } from '@axona/protocol';
```

```
const ephemeral = await createAuthorIdentity();              // unlinkable, one session
const durable   = await createAuthorIdentity({ persistAs: 'me' }); // recognizable across
↪  reloads
console.log(durable.authorId);   // 64-hex Author ID — share this as `owner`
```

`dumpIdentity(identity) → Promise<IdentityEnvelope>` *(advanced)*

Serialize a **node** identity to a JSON-safe envelope (the `privateKey` is exported as base64 PKCS#8). Loses the in-memory `CryptoKey` handle; reconstruct with `loadIdentity`.

```
const env = await dumpIdentity(node);
// { id, pubkey, privkey, region: { lat, lng }, createdAt, pow }
localStorage.setItem('node-identity', JSON.stringify(env));
```

Throws `IdentityError(IDENTITY_LOAD_FAILED)` if the key can't be exported.

`loadIdentity(envelope) → Promise<Identity>` *(advanced)*

Inverse of `dumpIdentity`. Reconstructs the full node `Identity`, verifying that (a) the stored id matches the freshly-derived id and (b) the private key corresponds to the public key (a sign→verify probe).

```
const env = JSON.parse(localStorage.getItem('node-identity'));
const node = await loadIdentity(env);
```

Throws `IdentityError`:

- `IDENTITY_INVALID_FORMAT` — malformed envelope, id mismatch, or private/public key mismatch.
- `IDENTITY_LOAD_FAILED` — PKCS#8 import failed.

> Author-identity persistence is handled inside `createAuthorIdentity({ persistAs })` — you don't dump/load author identities by hand. `dumpIdentity`/`loadIdentity` are for node identities (or when you manage your own storage layer).

`computeNodeId(pubkeyBytes, lat, lng) → Promise<string>` *(advanced)*

`computeNodeIdBigInt(pubkeyBytes, lat, lng) → Promise<bigint>` *(advanced)*

Derive (or verify) a 264-bit nodeId from a raw public key + region. `computeNodeId` returns the 66-hex form; `computeNodeIdBigInt` returns the BigInt. Used to validate a claimed nodeId against a pubkey + region without minting a fresh identity.

```
const id = await computeNodeId(node.pubkey, 38.0, -77.0);
console.log(id === node.id);   // true
```

---

3. AxonaPeer construction + lifecycle

`connect({ bridge, location, author?, k?, ready?, transport?, nodeIdentity?, web? }) → Promise<{ peer, author, nodeIdentity, transport, status, disconnect }>` *(kernel 4.16.0)*

The one-call bootstrap — import from the `connect.js` subpath:

```
import { connect } from '@axona/protocol/connect.js';
const { peer, author, status, disconnect } = await connect({
  bridge: 'wss://testnet.axona.net', location: { lat: 38, lng: -77 } });
```

Mints the connection identity (and, by default, an ephemeral author), builds the web transport + `NeuronNode` + `AxonaDomain` + `AxonaPeer`, starts everything, and awaits `peer.ready()`. Pure sugar over the constructors below — use them directly for custom wiring.

Param	Type	Notes
`bridge`	string	`wss://` bridge URL. Required unless `transport` injected.
`location`	`{ lat, lng }`	The node's real location. Required unless `nodeIdentity` injected.
`author`	`true` | string | identity | `false`	`true` (default) ephemeral author; a string → durable via `persistAs`; an author identity → used as-is; `false` → none.
`k`	number (20)	Routing closest-set size.
`ready`	object | `false`	Forwarded to `peer.ready()`; `false` skips the wait (`status` is `null`).
`transport` / `nodeIdentity`	—	Injection for tests, sim, custom stacks; the web transport is only loaded when no `transport` is given.
`web`	object	Extra options forwarded to the `webTransport` factory.

`disconnect()` performs `peer.leave()` + `peer.stop()` + `transport.stop()`, best-effort. `connect` lives outside the main barrel (the barrel stays environment-neutral); the import path is `@axona/protocol/connect.js`.

`AxonaPeer` is the per-node DHT contract implementation — one instance per running node.

`new AxonaPeer({ nodeIdentity, domain, transport, ... })`

Param	Type	Notes
`node`	`NeuronNode`	**required** — the per-node state object (`{ id, alive, transport, synaptome }`).
`nodeIdentity`	`Identity`	the connection key (from `createNodeIdentity`). Used for the handshake, routing, and signing the node's own actions. It **never** signs a publish.
`transport`	`Transport`	the transport instance (web/sim/node). The peer uses it for `send`/`notify`/`lookup`.
`domain`	`AxonaDomain`	shared routing parameters. **`domain` or `engine` is required.**
`engine`	object	legacy simulator engine; pass instead of `domain` only in sim/tests.
`axonaManager`	`AxonaManager`	pre-built pub/sub manager; if omitted, resolved on first `pub`/`sub`.
`persist`	`PersistenceAdapter`	enables auto-checkpointing of identity, subscriptions, synaptome, and hosting state.
`maxPublishBytes`	`number`	per-publish cap; clamped to the WebRTC-interop floor (16 KiB) and never above the absolute ingress cap. Override only for known-homogeneous fleets.

Param	Type	Notes
`synaptomeMaintain`	`boolean | { kNear?, intervalMs?, maxPerTick? }`	*(advanced; v4.9.0)* opt into continuous **near-quota maintenance** — the peer keeps its `kNear` (≈5) XOR-nearest "successor" links filled through churn so greedy routing's last mile stays complete. `true` uses the defaults `{ kNear: 5, intervalMs: 15000, maxPerTick: 3 }`. Off by default; the standard `axona-peer`/relay builds set it. Long-range "fingers" are handled separately by the routing anneal.

The production path is `{ nodeIdentity, domain, transport }`. The `{ engine }` / `{ axonaManager }` forms are for the simulator and tests.

```
import { AxonaPeer, AxonaDomain, NeuronNode, createNodeIdentity } from '@axona/protocol';

const node = await createNodeIdentity({ lat: 38, lng: -77 });
const peer = new AxonaPeer({
  nodeIdentity: node,
  domain:       new AxonaDomain({ k: 20 }),
  node:         new NeuronNode({ id: BigInt('0x' + node.id), lat: 38, lng: -77 }),
  transport,
});
```

Throws plain `Error` if `node` is missing, or if neither `engine` nor `domain` is supplied.

`peer.start() → Promise<void>`

Bring the peer up — installs wire-level routing handlers, the delivery hook, the peer-bound/peer-died eviction handlers, and (if `persist` is wired) hydrates persisted state. Idempotent.

`peer.ready({ minPeers?, timeoutMs?, stableMs?, pollMs? }) → Promise<{ ready, peers, ms, reason }>`

Wait for the routing mesh to warm up before you `pub`/`sub`. `start()` returns as soon as local state is set up, but mesh handshakes complete asynchronously over the next few seconds; publishing/subscribing on a cold synaptome routes to too few root axons. `ready()` resolves once the synaptome reaches `minPeers`, or — failing that — settles at a **stable non-zero plateau** (unchanged for

`stableMs`), or the `timeoutMs` budget elapses. Use it instead of hand-rolling a `peer.peers().length` / `node.synaptome.size` poll.

Param	Type	Default	Notes
`minPeers`	`number`	`4`	Resolve `ready:true` as soon as the synaptome reaches this size.
`timeoutMs`	`number`	`10000`	Upper bound; on expiry resolves with `ready` = (peers > 0).
`stableMs`	`number`	`1500`	If below `minPeers` but non-zero and unchanged this long, resolve `ready:true` (`reason:'stable'`).
`pollMs`	`number`	`150`	Synaptome poll interval.

Returns `{ ready:boolean, peers:number, ms:number, reason:'minPeers'|'stable'|'timeout' }`. Never rejects — a cold/isolated start resolves `{ ready:false, …, reason:'timeout' }` so you can surface "still connecting" rather than throw.

```
await peer.start();
const status = await peer.ready();          // defaults: 4 peers or a stable plateau, ≤10s
if (!status.ready) console.warn('mesh still warming — publishes may be thin');
```

`peer.stop() → Promise<void>`

Tear down: release event listeners and the peer-lifecycle hooks. Idempotent. (For a graceful network exit, prefer `peer.leave`.)

`peer.join(sponsor?) → Promise<void>`

Bootstrap into the mesh. Calls `start()` first if needed, brings up the transport, and — if a `sponsor` is given — opens a channel to it and seeds the synaptome.

Param	Type	Notes
`sponsor`	`string`	66-char hex nodeId of a known peer. Omit to start standalone (inbound connections will populate the synaptome).

```
await peer.join();                // standalone; wait for inbound peers
await peer.join(bridgeNodeIdHex); // open a channel to a sponsor and seed
```

Throws `TransportError`:

- `TRANSPORT_NOT_STARTED` — `transport.start` failed, or `sponsor` is not 66-char hex.
- `TRANSPORT_PEER_UNREACHABLE` — the sponsor isn't reachable.

`peer.leave({ drain?, notify?, timeoutMs? }) → Promise<void>`

Graceful shutdown.

Param	Type	Default	Notes
`notify`	`boolean`	`true`	send a `peer-leaving` notification to every synaptome peer so they drop us proactively (sub-second handoff instead of heartbeat timeout).
`drain`	`boolean`	`true`	pause briefly for in-flight publishes/pulls to settle before closing.
`timeoutMs`	`number`	`5000`	bound on the drain wait.

Closes the transport last, force-flushes persistence, and stops listeners.

```
await peer.leave({ drain: true, notify: true, timeoutMs: 3000 });
```

`peer.getNodeId() → string | bigint`

Returns whatever was passed as `node.id` at construction time (BigInt if you followed the convention).

`peer.snapshot() → Promise<object>`

Serialize this peer's state to a JSON-safe envelope: `{ formatVersion: '1.0', snapshotAt, wireVersion, identity, synaptome, subscriptions }`. Store it however you like (encrypted, synced, custom DB).

```
const state = await peer.snapshot();
localStorage.setItem('peer-state', JSON.stringify(state));
```

`AxonaPeer.fromSnapshot(state, opts) → Promise<AxonaPeer>` ***(static)***

Reconstruct a peer from a `snapshot()` envelope. The returned peer is constructed with identity / synaptome pre-loaded, but the transport is **not** started — call `peer.join(sponsor?)` to bring it up. Subscription **handlers are not restored** (functions don't serialize); the restored list is exposed at `peer.pendingSubscriptions` so you can re-register them with `peer.sub`.

```
const state = JSON.parse(localStorage.getItem('peer-state'));
const peer  = await AxonaPeer.fromSnapshot(state, { domain, node, transport });
for (const s of peer.pendingSubscriptions) {
  await peer.sub(s.topic, onMsg, { since: s.since });
}
await peer.join();
```

Throws `TypeError` / `RangeError` if `state` isn't a `1.0`-format snapshot object.

---

4. Pub/sub

4.1 Topic addressing

(Folded in from the former standalone "Topic IDs" note.)

A topic is not a bare string — it's a small **descriptor**, and the topic ID is a hash of that descriptor with a one-byte region prefix:

```
topicId = regionByte || SHA-256(canonical({ owner, name, write }))   // 33 bytes = 66 hex chars
```

Field	Meaning
`region`	a real geographic cell; becomes the leading byte of the ID
`owner`	an **Author ID** (a publish key), or absent
`name`	the human label — `"lobby"`, `"profile"`
`write`	`"open"` or `"owner"` — defaults by whether `owner` is set

Because the ID is a pure function of those fields, anyone who knows the fields computes the identical ID — no registry, no coordination.

The `region` field accepts a name, hex string, decimal string, or number — all normalize to the same region byte:

```
{ region: 'useast', name: 'lobby' }   // name
{ region: '0x89',   name: 'lobby' }   // hex string   --\  same topic ID
{ region: 137,      name: 'lobby' }   // number       --/  (leading byte 0x89)
```

Omit `region` and it defaults to the publisher's own node region (top byte of its node ID). Unknown regions throw `RangeError`.

`write` defaults by whether there's an `owner`:

Descriptor	Resolved `write`	Meaning
`{ region, name }` (no owner)	`open`	open lobby — anyone publishes
`{ region, owner, name }` (no write)	**`owner`**	owned — only the owner publishes
`{ region, owner, name, write:'owner' }`	`owner`	same ID as the row above
`{ region, owner, name, write:'open' }`	`open`	owner-namespaced, anyone publishes (inbox/wall)
`{ region, name, write:'owner' }` (no owner)	`open`	no owner -> `write` ignored

Two rules: **no `owner` -> the topic is open and `write` is ignored**; **an `owner` -> `write` defaults to `'owner'`** (the safe default — forgetting `write` can never silently make an owned feed world-writable).

Share the ID for reading; share the descriptor for writing. `sub`, `pull`, and `metrics` accept the 66-hex ID or a descriptor. `pub` (and the owner op `kill`) requires the descriptor — a bare ID is rejected, because a hash can't reveal its `owner`, so the ID alone can't prove write authorization.

`deriveTopicId(descriptor, selfRegion?)` → `Promise<string>` Compute the 66-hex topic ID for a descriptor — the same function the peer uses internally. This is what you hand to someone so they can subscribe.

Param	Type	Notes
`descriptor`	`TopicDescriptor`	`{ region?, owner?, name, write? }`.
`selfRegion`	`string | number`	fallback region when `descriptor.region` is omitted (the publisher's node region).

```
import { deriveTopicId } from '@axona/protocol';

const lobbyId = await deriveTopicId({ region: 'useast', name: 'lobby' });
const feedId  = await deriveTopicId({ region: 'useast', owner: me.authorId, name: 'profile'
↪  });
// → "89…"  (66 hex chars; leading "89" is the region byte) — share out-of-band
```

Throws `TypeError` (empty `name`), `RangeError` (unknown region, bad `owner` format, or no region and no `selfRegion`).

`deriveTopicIdBig(descriptor, selfRegion?)` → `Promise<bigint>` BigInt variant of `deriveTopicId` (the form kernel internals hold). Same parameters and errors.

resolveTopic(descriptor, selfRegion?) → Promise<object> (advanced) The full resolver behind `deriveTopicId`. Returns the normalized descriptor plus the id: `{ region (code), owner, name, write, topicId }`. Use it when you need the resolved `write`/`region`, not just the id.

```
const r = await resolveTopic({ region: 'useast', owner: me.authorId, name: 'profile' });
// { region: 137, owner: '<64-hex>', name: 'profile', write: 'owner', topicId: '89…' }
```

canonical(value) → string (advanced) Stable, total, JSON-valid canonical encoding (object keys sorted at every level). Two semantically-identical values canonicalize to the same bytes across runs and implementations — the basis of content-addressed `msgId`s and signature stability.

sha256Hex(input) → Promise<string> (advanced) `SHA-256` of a string or `Uint8Array`, returned as lowercase hex.

4.2 publish / subscribe

peer.pub(topic, message, { signWith }) → Promise<string> Publish a message to a topic. **Returns** the 64-char hex `msgId`.

Arg	Type	Notes
`topic`	`TopicDescriptor`	**must be a descriptor** — a bare id is rejected (the storing node needs the descriptor to verify the write policy).
`message`	`any`	JSON-serializable payload.
`opts.signWith`	`AuthorIdentity | ANONYMOUS`	**required** — name a signer, or pass the `ANONYMOUS` sentinel to publish unsigned. There is no default author.

```
import { createAuthorIdentity, ANONYMOUS } from '@axona/protocol';
const me = await createAuthorIdentity();

const id = await peer.pub({ region: 'useast', name: 'lobby' }, { text: 'hi' }, { signWith: me
↪  });
await peer.pub({ region: 'useast', name: 'lobby' }, 'anon msg', { signWith: ANONYMOUS });
```

For an **owned** topic, `signWith` must be the owner key — publishing with any other key throws before it leaves the peer.

Maximum size. The cap is on the *serialized envelope* and defaults to the WebRTC-interoperable reliable-delivery floor (16 KiB) so any peer on any path can receive it. Chunk larger payloads (`@axona/protocol/std/chunk`) or publish a content-reference and transfer bytes out-of-band.

Chunking is reliable by default (`std/chunk`, kernel ≥ 3.6.0). `publishChunkedBytes(peer, bytes, { topic, signWith, name, mime })` splits a payload into floor-sized messages, publishes them, then

verifies what the mesh actually cached and re-publishes any gaps — so a *reload* subscriber can reassemble from replay. `peer.pub` is fire-and-forget into the transport buffer, so a fast burst would otherwise drop chunks before they cache; the verify+repair pass means you do **not** hand-tune a publish throttle. `receiveChunkedBytes(peer, topic, { timeoutMs })` reassembles and resolves with `{ bytes, name, mime, size, meta }`, or **rejects** (naming the missing indices) on timeout — it never hangs. Files are capped to the per-topic replay-cache ceiling so you never create a transfer a reload joiner can't complete.

Delivery (4.x). A publish routes to the topic's K-closest **cohort** and is replicated across it (not a single root), so it survives the closest node churning out. A publish is still one-shot fire-and-forget — there is **no delivery ack to the publisher** (the transport identity is deliberately unlinked from the author identity) — but two automatic mechanisms cover the gaps: a freshly-joined ("cold") node re-sends its first publishes a few times over ~1 s (v4.11.0) so a not-yet-warm routing table doesn't strand them, and a background retry re-sends a recent publish toward the true root until the publisher observes its own `msgId` land. You do not tune any of this. Since v4.22.0 the cohort also converges to the **union** of its members' history across root transitions, so a `since:'all'` subscriber attaching right after the root moved still replays the full timeline, not just the newer half.

Throws `PublishError`:

- `PUBLISH_INVALID_TOPIC` — a bare id was passed, or the topic isn't a `{ name, … }` object.
- `TOPIC_REGION_REQUIRED` — no region named and the peer has no node region to default to.
- `PUBLISH_NO_PUBLISH_IDENTITY` — no `signWith` given.
- `WRITE_POLICY_VIOLATION` — owned topic, signer is not the owner.
- `PUBLISH_SIGN_FAILED` — `signWith` lacks `privateKey`/`pubkeyHex`, or signing failed.
- `PUBLISH_INVALID_MESSAGE` — message isn't JSON-serializable.
- `PUBLISH_PAYLOAD_TOO_LARGE` — enveloped message exceeds the cap.

`peer.sub(topic, handler, { since? }) → Promise<Subscription>` Subscribe. `handler` is invoked with the full `Envelope` for each delivery.

Arg	Type	Notes
`topic`	`TopicDescriptor | string`	a descriptor **or** a 66-hex topic ID (the shareable read handle).
`handler`	`(envelope) => void`	called per delivery; also receives retraction markers `{ deleted: true, msgId, topic }`.
`opts.since`	`'all' | 'latest' | number`	replay control (below).

`since` modes:

`since`	What you get
omitted / `undefined`	live tail only — future messages.
`'latest'`	the most recent cached message, then live tail.
`'all'`	everything in the replay cache, then live tail.

since	What you get
`<number>`	messages newer than the timestamp (ms epoch), then live tail.

Returns a `Subscription`; call `.stop()` to cancel.

```
const sub = await peer.sub({ region: 'useast', name: 'lobby' }, (env) => {
  if (env.deleted) { dropFromUI(env.msgId); return; }
  console.log(env.signerPubkey, env.message);
}, { since: 'all' });

// or subscribe by shared id:
const sub2 = await peer.sub(lobbyId, onMsg, { since: 'latest' });

await sub.stop();
```

Throws `SubscribeError(SUBSCRIBE_HANDLER_MISSING)` (handler not a function); `PublishError(PUBLISH_INVALID_TOPIC)` (a string that isn't a valid 66-hex id).

`peer.unsub(topic)` → `Promise<{ ok, removed }>` Unsubscribe by topic — the counterpart to `sub`. Stops **every** local subscription this peer holds for the topic (no need to keep the handle) and, once the last one goes, sends the network unsubscribe so the topic's roots drop this peer. Self-only by construction. Idempotent — returns `{ ok: true, removed: 0 }` for a topic you're not on.

Takes a **descriptor** (it derives the topicId the same way `sub`'s descriptor form does).

```
const { removed } = await peer.unsub({ region: 'useast', name: 'lobby' });
```

4.3 pull / metrics

`peer.pull(msgId, { topic, timeoutMs? })` → `Promise<Envelope | null>` Fetch one message from the topic's replay cache. Returns `null` on a cache miss (including a message that aged out of the hold window) — that's expected, not an error.

Arg	Type	Notes
`msgId`	`string | null`	64-char hex, **or `null`/omitted for the topic's most-recent message**.
`opts.topic`	`TopicDescriptor | string`	descriptor **or** 66-hex id.
`opts.timeoutMs`	`number`	default `1000`.

Nearest-replica reads (v4.11.1+). A pull is answered by the **first replica the request reaches** — a cohort member, a child relay, or a `host()` node — not necessarily the topic's root.

This lowers latency, spreads reads off the root, and lets a pull that would otherwise strand toward the root be served by any cache-holder it passes. Two consistency tiers:

- **`pull(msgId)`** — *exact.* `msgId = H(publisher‖message)`, so a nearer copy **is** the copy; you always get that specific immutable message (or `null`).
- **pull-latest** (`msgId` null) — *recent, eventually-consistent.* You get whatever newest that first replica holds, which may be a beat behind the root's very newest until the cohort converges. This is deliberate: it keeps a heavily-polled "current value" read from making the root a throughput bottleneck. If you need the linearizable newest, pull a specific `msgId`.

A cache-holding replica has not tombstoned the message (a `kill` drops it from cache), so a pull never returns a killed message. A successful pull slides the message's hold time forward (bounded by the 48 h ceiling).

```
const env    = await peer.pull(msgId, { topic: feedId });
const latest = await peer.pull(null, { topic: feedId });   // most recent on the topic
```

Throws `PullError`:

- `PULL_INVALID_MSGID` — a non-null `msgId` that isn't 64-char hex.
- `PULL_AXONS_UNREACHABLE` — the manager can't service the request.

`peer.metrics(topic, { timeoutMs? }) → Promise<metricsObj>` A one-shot read of a topic's latest published metric snapshot — for **any** topic, open or owned. Metrics are **demand-driven** (v4.12.0): subscribing to `metricTopic(T)` — which `metrics()` does internally — sends a renewable *metrics-on* lease toward the data topic's root, and **any node that roots the topic** (relay or ordinary client peer alike) then publishes a signed snapshot to that *open* metric topic every ~20 s while at least one metric subscriber remains. `metrics()` collects whatever arrives (or is replayed from the cache) during its short window and returns the freshest view. Owned topics' metrics are **public too** — anyone who can derive the id can read them — so there is no owner gate.

> **Cold-topic note.** Publication starts *on demand*, and since v4.16.1 the root answers a freshly armed lease **immediately** — the first snapshot rides back at routing latency (~0.3 s measured on testnet), normally inside the default 1500 ms collection window. Under churn (the root re-homing at that instant) it can take a few seconds; if the window closes empty the call returns `stale: true` — retry, widen the window, or — better — keep a standing `sub(metricTopic(T), …)` and treat metrics as the stream it is. If *any* peer watched the topic's metrics within the 48 h hold window, replay serves the last cached snapshot immediately regardless.

Cohort-aware (v4.10.1). Under the K-closest cohort model every co-hosting root publishes its own snapshot, so `metrics()` collects them over the window and **aggregates**: `subscribers` is **summed** (each root reports only its own subscriber subset, so the sum is the topic-wide total), while `current_count`, `seq`, and `bytes` are **maxed** (they converge across the cohort via anti-entropy; max tolerates a lagging member). `cohortSize` tells you how many roots reported.

Arg	Type	Notes
`topic`	`TopicDescriptor | string`	the **data** topic — descriptor **or** 66-hex id.

Arg	Type	Notes
`opts.timeoutMs`	`number`	how long to collect cohort snapshots; default `1500`.

Returns:

```
{
  current_count: number;       // live (non-expired, non-killed) messages retained now — max
 ↪  across cohort
  seq:           number;       // dense message counter: monotonic high-water of total events
 ↪  ever
                               //   emitted on the topic (kills included) — max across cohort
  subscribers:   number;       // topic-wide subscriber total — summed across the cohort
  bytes:         number;       // live cached envelope bytes — max across cohort
  publishes:     number;       // present only if the publisher tracks it; else 0
  ts:            number|null;  // freshest snapshot timestamp
  signer:        string|null;  // the freshest snapshot envelope's signerPubkey (provenance)
  cohortSize:    number;       // # of distinct roots that reported a snapshot
  stale:         boolean;      // true ⇒ no snapshot seen (no metrics publisher roots this
 ↪  topic)
}
```

Use `seq` for total-ever (published + killed), `current_count` for currently-live, and their gap to spot churn. `subscribers` is a topic-wide count for UX, not billing. All values are advisory.

```
const m = await peer.metrics({ region: 'useast', name: 'lobby' });
if (!m.stale) console.log(`${m.subscribers} subscribers, ${m.current_count} live (${m.seq}
 ↪  total), ${m.cohortSize} roots`);
```

> **For a live dashboard, prefer `sub(metricTopic(T), …)` directly** (below) — one standing subscription gives you the latest snapshot plus a rolling history. `metrics()` is the convenience one-shot. Trust is **advisory**: the metric topic is open, so check `signer` if you need provenance (pin to a known relay key).

`metricTopic(dataTopicId)` → `TopicDescriptor` (subscribe to metrics — the live path) The producer side of the convention. Compute the derived, open metric topic with `metricTopic()` and `sub()` it — **your subscription is what turns publishing on**: it routes a renewable *metrics-on* lease to the data topic's root, and while the lease is fresh the root publishes a signed snapshot every ~20 s, for **both open and owned** data topics. You get the latest snapshot via replay-on-subscribe plus every update — one subscription instead of a poll — and, because snapshots are ordinary messages that age out at the 48 h hold ceiling, a **rolling ~48 h history for free** to plot trends. This is also how you subscribe to an **owned** topic's metrics without owning it.

When snapshots arrive. The timing contract (since v4.16.1):

- **First snapshot: at routing latency.** The root answers the moment your subscription's lease

arms — **~0.3 s measured on testnet**, arriving with (or before) your data-topic replay, whether or not anyone has ever published or watched. Allow a few seconds if the root churns at exactly that moment (it must re-home first). If any peer watched this topic's metrics within the last 48 h, replay hands you the most recent snapshot immediately as well.

- **Cadence:** one snapshot every **~20 s** per rooting node, for as long as at least one metric subscriber remains.
- **Shut-off:** the lease lapses **~70 s** after the last metric subscriber unsubscribes; publishing stops until demand returns.
- **Silence is *unknown*, not zero.** Until the first snapshot (or your data replay) arrives, don't render "no activity" as a definitive answer; a `current_count: 0` snapshot is the real "nothing here."

```
import { deriveTopicId, metricTopic } from '@axona/protocol';

const id = await deriveTopicId({ region: 'useast', name: 'lobby' });
await peer.sub(metricTopic(id), (env) => {
  const m = JSON.parse(env.message);
  // { topic, ts, by, signer, current_count, seq, subscribers, bytes }
  render(m.subscribers, m.current_count);
}, { since: 'all' });   // since:'all' → latest snapshot + the rolling history
```

name	returns	notes
`metricTopic(dataTopicId)`	`TopicDescriptor`	open topic `{ region, name: 'axona:metric:' + id }`. Pass the **resolved 66-hex** id (`await deriveTopicId(desc)`), not the descriptor. Region byte is inherited from the data topic.
`isMetricTopic(descriptor)`	`boolean`	true if a topic is itself a metric topic (the recursion guard a relay uses).
`isMetricTopicName(name)` / `dataTopicIdOf(descriptor)`	`boolean` / `string|null`	name-prefix test; inverse (metric topic → data id).
`METRIC_NAMESPACE`	`string`	the frozen reserved prefix `'axona:metric:'`.

Trust is **advisory**: the metric topic is open (anyone may publish to it), so treat a snapshot as a hint — if you need to, pin trust to a known relay's `env.signerPubkey`. The protocol does not prove a snapshot is authoritative.

4.4 owner + creator ops: kill

These take a **descriptor** (not a bare id) and a signer via `{ signWith }`.

peer.kill(topic, msgId, { signWith }) → Promise<{ ok }> Retract a message you published — "unsend". Authorized by **authorship**: the roots accept the kill only if its signer matches the signer of the cached message, so you can only kill messages **you** signed. The kill is distributed to the topic's **K-closest cohort** (v4.10.0), not one root: every cohort member drops it from its replay cache, tombstones the `msgId`, and every internal history transfer (hand-off, catch-up, backup replication) carries that tombstone — so a late subscriber attaching to any cohort member can't be served the killed copy. Subscribers get a delete marker (`{ deleted: true, msgId, topic }`).

Arg	Type	Notes
`topic`	`TopicDescriptor`	the topic the message was published to.
`msgId`	`string`	**required** 64-char hex (the value `pub` returned).
`opts.signWith`	`AuthorIdentity`	the **same author key** that published the message.

```
const id = await peer.pub({ region: 'useast', name: 'lobby' }, { text: 'oops' }, { signWith:
↪  me });
await peer.kill({ region: 'useast', name: 'lobby' }, id, { signWith: me });
```

Best-effort redaction, not a cryptographic un-send: a subscriber who already has the plaintext can keep it; an anonymous message has no provable creator and **cannot** be killed.

Throws `KillError`: `KILL_INVALID_MSGID` (bad msgId), `KILL_SIGN_FAILED` (no/invalid `signWith`, or signing failed). A network that can't authorize the kill is **not** an error — it just won't take effect.

Removed in v4.3.0: `peer.unpub()` and `peer.touch()`. `kill(topic, msgId)` is now the single retraction primitive — retract individual messages you signed. (`unpub` cleared a whole owned-topic queue; `touch` was a hold-time keep-alive. Both are gone — a topic's queue empties naturally as messages age out at the 48 h hold ceiling, and a still-relevant message is kept current by re-publishing.) `touch()` remains callable as a no-op for source compatibility but does nothing; do not use it.

4.5 hosting: host / unhost

peer.host(topic?, opts?) → Promise<{ ok, scope, topicId? }> Store and serve a topic for other peers **without subscribing** — the relay / infrastructure primitive. The node becomes a willing root/replica (publishes land on it, subscribers pull replays from it) but registers **no handler** and delivers nothing to a local app.

- `peer.host()` — host this node's **own keyspace neighborhood**: get recruited as a root for whatever topics land near this node's id ("host whatever lands near me"). The zero-config relay mode; returns `{ ok: true, scope: 'keyspace' }`.
- `peer.host(topic)` — host one specific topic (descriptor). Returns `{ ok: true, scope: 'topic', topicId }`.

Wire-compatible with every kernel (reuses `subscribe-k`), respects the proximity gate, and is idempotent.

```
await peer.host();                                          // headless relay
await peer.host({ region: 'useast', name: 'pow-results' }); // host one topic
```

peer.unhost(topic?, opts?) → Promise<{ ok, scope }> Counterpart to `host`. `unhost()` turns off keyspace hosting; `unhost(topic)` drops one hosted topic. Does **not** touch your subscriptions. Idempotent.

peer.rootedTopics() → Array<{ topicId, descriptor, current_count, seq, subscribers, bytes }> (infra) Synchronous, local-only introspection of the topics this node currently **roots** — each with its signed topic descriptor (recovered from a cached envelope, or `null` for an empty/cold role) and a locally-computed snapshot (`current_count`, `seq`, this member's `subscribers` subset, `bytes`). No network (unlike `metrics()`). This is the read side that powers the relay metric-publish loop (§4.3): walk it, skip `isMetricTopic(d)` and non-open descriptors, and `pub(metricTopic(topicId), …)` for the rest. Returns `[]` on a routing-only peer with no `AxonaManager`.

4.6 topic limits

Every topic's replay queue is bounded:

- **Max messages** — up to `CACHE_MAX` (**1024**) per topic, also bounded by a 16 MB byte ceiling on the cache. When full, the lowest-ordered message (by signed `seq`) is evicted deterministically. (Configurable per peer via the `replayCacheSize` constructor option; a value of 1 is a *retained/latest-value* slot — pair it with `peer.pull(null, …)`.)
- **Hold time** — default 24 h, hard ceiling 48 h. A message past its hold is swept (stops being delivered or pulled). A `pull` or `touch` slides the hold forward, bounded by the ceiling.
- **Per-publisher quota (open topics only)** — one publisher occupies at most ¼ of the queue, so a flooder can't evict everyone else. Owner-gated topics aren't quota'd.

---

5. Direct messaging

1:1 messages between specific peers, bypassing pub/sub. `targetId` is the 66-char hex nodeId; the peer must already be in the synaptome.

`peer.send(targetId, message) → Promise<reply>`

Request/response. Resolves to whatever the recipient's `onMessage` handler returns.

```
const reply = await alice.send(bobIdHex, { kind: 'ping', body: 'hi' });
```

Throws `TypeError` if `targetId` isn't 66-char hex.

`peer.notify(targetId, message) → Promise<void>`

Fire-and-forget. Resolves once enqueued, not when delivered.

```
await alice.notify(bobIdHex, { kind: 'typing', user: 'alice' });
```

Throws `TypeError` if `targetId` isn't 66-char hex.

`peer.onMessage(handler) → void`

Register the single inbound direct-message handler. Calling it again **replaces** the previous handler. Signature `(senderId, message) => reply | void`: a returned value resolves the sender's `send()`; for `notify()` callers the return is discarded.

```
peer.onMessage(async (senderIdHex, message) => {
  switch (message?.kind) {
    case 'ping':   return { reply: 'pong' };   // resolves send()
    case 'typing': console.log(senderIdHex, 'typing'); return;  // notify discards
  }
});
```

`senderId` is the transport-authenticated sender, surfaced as 66-hex.

---

6. Mesh introspection + events

`peer.peers() → string[]`

Current synaptome membership as 66-char hex strings.

`peer.onPeerJoin(handler) → () => void`

Fires when a peer is admitted to the synaptome. Handler receives `(peerIdHex, event)`. Returns an unsubscribe function.

```
const off = peer.onPeerJoin((id, ev) => console.log('admitted', id, ev?.addedBy));
```

`peer.onPeerLeave(handler) → () => void`

Symmetric — fires on eviction (TTL, churn, explicit unbind). Same signature.

`peer.lookup(targetKey) → Promise<lookupResult>`

Iterative XOR-routed walk to find `targetKey` (typically a peer ID, but any 264-bit key works). `targetKey` is a BigInt.

```
const r = await peer.lookup(BigInt('0x' + targetHex));
// { found: boolean, hops: number, time: number /* ms */, path: bigint[] }
```

Returns `null` if the node isn't alive.

`peer.health() → healthObj`

A cheap synchronous diagnostic snapshot — safe to poll on a UI tick:

```
peer.health();
// {
//   nodeId, synaptomeSize, peers: [...], subscriptions,
//   axonRoles: [{ topic, isRoot, children, cacheSize }],
//   hosting:   { keyspace, topics } | null,
//   wireVersion, started,
//   transport: { boundCount, meshChannels, meshOpen, meshBound, bridgeState } | null,
//   meshDegraded: boolean,
// }
```

`meshDegraded === true` (open channels materially exceed authenticated binds) is the routing-truth signal — a single tick can be a mid-handshake transient; a value that stays `true` across polls is the real warning. `transport` is `null` on sim/node transports, populated on web.

`peer.onLog(level, handler) → () => void`

Subscribe to structured log events. **The level comes first** (`'debug' | 'info' | 'warn' | 'error'`); handler is `(msg, context?) => void`. Returns an unsubscribe function.

```
const off = peer.onLog('warn', (msg, ctx) => console.warn(msg, ctx));
```

Throws `TypeError` on an invalid level or non-function handler.

`peer.onError(handler) → () => void`

Fires on background `AxonaError` emissions (transport failures during heartbeat, persistence warnings) the kernel surfaces asynchronously rather than throwing. Returns an unsubscribe function.

`peer.onUpgradeRequired(handler) → () => void`

Fires on a wire-version handshake mismatch, with the `UpgradeRequiredError` carrying `{ reason, serverVersion, clientVersion, downloadUrl }`.

```
peer.onUpgradeRequired((err) => showUpgradeBanner(err.context.downloadUrl));
```

---

7. Envelopes + verification

The kernel builds and verifies envelopes for you inside `peer.pub` / `peer.sub`. These low-level helpers are for verifying what you consume, signing a DM payload by hand, or testing.

`buildEnvelope({ topic, message, ts?, seq?, identity, sign? }) → Promise<Envelope>`

Param	Type	Notes
`topic`	`TopicDescriptor`	the descriptor — signed, so the signature binds the exact write policy.
`message`	`any`	JSON-serializable.
`ts`	`number`	ms timestamp; defaults to `Date.now()`.
`seq`	`number`	per-publisher monotonic sequence (default `0`; `peer.pub` supplies a real value).
`identity`	`AuthorIdentity`	required when `sign: true`.
`sign`	`boolean`	default `true`.

Returns an `Envelope`. The signature covers a domain-tagged core (`axona:pubsub-envelope:v2`).

Throws `TypeError` if `identity` lacks `privateKey`/`pubkeyHex` when `sign: true`.

`verifyEnvelope(envelope) → Promise<{ ok, reason?, signed }>`

Verify the signature against the domain-tagged `(seq, ts, topic, message)` core and that `msgId` matches. **Returns a result object, not a boolean** — check `.ok`.

```
const res = await verifyEnvelope(env);
if (!res.ok) console.warn('forged/invalid', env.msgId, res.reason);
// res.signed === false for a valid unsigned envelope (res.ok === true)
```

A common mistake: `if (!(await verifyEnvelope(env)))` is always false — the return value is a truthy object. Always test `.ok`.

`reason` (when `!ok`) is one of `not_an_object`, `missing_msgId`, `missing_ts`, `missing_topic`, `missing_message`, `missing_seq`, `missing_signerPubkey`, `unknown_signature_scheme`, `wrong_key_or_signature_length`, `bad_signature`, `bad_msgid`. `verifyEnvelope` checks signature + msgId but **not** freshness (the legitimate replay path serves cached history).

`computeMsgId({ publisher?, message }) → Promise<string>`

Stand-alone msgId computation matching `buildEnvelope`: `sha256(canonical({ publisher, message }))`, where `publisher` is the signer's pubkey (or `null` for anonymous). Time and seq are deliberately not folded in.

```
const msgId = await computeMsgId({ publisher: env.signerPubkey, message: env.message });
console.log(msgId === env.msgId);   // true
```

`checkFreshness(envelope, { now?, maxSkewMs? }) → { ok, reason? }`

Test whether an envelope's signed `ts` is within the freshness window (`MAX_PUBLISH_SKEW_MS`, 5 min by default). Live-publish ingress enforces this in the kernel; call it yourself only if you need the check on a path the kernel doesn't gate.

Envelope constants

- `ENVELOPE_DOMAIN` — `'axona:pubsub-envelope:v2'` (signature domain tag).
- `MAX_PUBLISH_SKEW_MS` — `300000` (the freshness window).

---

8. Errors

All thrown errors inherit from `AxonaError`:

```
class AxonaError extends Error {
  code:     string;   // stable UPPER_SNAKE identifier — switch on this, not message
  cause?:   Error;
  context?: object;   // machine-readable details
}
```

Subclasses and their codes:

Class	Codes
`IdentityError`	`IDENTITY_KEYGEN_FAILED`, `IDENTITY_LOAD_FAILED`, `IDENTITY_INVALID_FORMAT`
`TransportError`	`TRANSPORT_NOT_STARTED`, `TRANSPORT_PEER_UNREACHABLE`, `TRANSPORT_TIMEOUT`, `TRANSPORT_CHANNEL_CLOSED`, `TRANSPORT_HELLO_FAILED`
`PublishError`	`PUBLISH_INVALID_TOPIC`, `PUBLISH_SIGN_FAILED`, `PUBLISH_NO_PUBLISH_IDENTITY`, `TOPIC_REGION_REQUIRED`, `WRITE_POLICY_VIOLATION`, `PUBLISH_REPLICATION_FAILED`, `PUBLISH_PAYLOAD_TOO_LARGE`, `PUBLISH_INVALID_MESSAGE`
`SubscribeError`	`SUBSCRIBE_INVALID_TOPIC`, `SUBSCRIBE_ATTACH_FAILED`, `SUBSCRIBE_HANDLER_MISSING`
`KillError`	`KILL_INVALID_TOPIC`, `KILL_INVALID_MSGID`, `KILL_SIGN_FAILED`
`UnpubError`	`UNPUB_INVALID_TOPIC`, `UNPUB_PUBLIC_TOPIC`, `UNPUB_SIGN_FAILED`
`TouchError`	`TOUCH_INVALID_TOPIC`, `TOUCH_INVALID_MSGID`, `TOUCH_SIGN_FAILED`
`PullError`	`PULL_INVALID_MSGID`, `PULL_AXONS_UNREACHABLE`

Class	Codes
`MetricsError`	`METRICS_AXONS_UNREACHABLE`
`UpgradeRequiredError`	`UPGRADE_REQUIRED` (bridge close code 4426)

The full taxonomy is in `ErrorCodes`:

```
import { ErrorCodes } from '@axona/protocol';
console.log(ErrorCodes.WRITE_POLICY_VIOLATION);  // 'WRITE_POLICY_VIOLATION'
```

Switch on `.code`, not `.message`:

```
try { await peer.pub(topic, msg, { signWith: me }); }
catch (err) {
  if (err.code === ErrorCodes.WRITE_POLICY_VIOLATION) { /* not the owner */ }
  else if (err.code === ErrorCodes.PUBLISH_NO_PUBLISH_IDENTITY) { /* name a signer */ }
  else throw err;
}
```

Wire-format helpers

Symbol	Notes
`isWireError(obj)` → `boolean`	does this look like an over-the-wire `AxonaError`?
`fromWire(obj)` → `AxonaError`	reconstruct the typed error (unknown classes fall back to `AxonaError`).
`err.toWire()` → `object`	`{ __axonaError: true, class, code, message, context }`.

Errors thrown inside a remote `onMessage` (reached via `peer.send`) survive serialization with their class and code intact.

---

9. Persistence

`PersistenceAdapter` (abstract class)

The storage contract. Implement it to plug in your own backend.

```
class PersistenceAdapter {
  async load(key)        { /* → previously-saved value or null */ }
  async save(key, value) { /* persist value */ }
  async delete(key)      { /* remove key */ }
}
```

Pass an instance as the `persist` option of the `AxonaPeer` constructor; the peer auto-checkpoints identity, subscriptions, synaptome, and hosting state.

`InMemoryPersistence`

Reference implementation (environment-neutral) used by tests and by the default `persist: false` path.

```
import { AxonaPeer, InMemoryPersistence } from '@axona/protocol';
const peer = new AxonaPeer({ nodeIdentity, domain, node, transport,
  persist: new InMemoryPersistence() });
```

`FilePersistence` / `IndexedDBPersistence` (sub-path imports)

Platform-specific implementations — sub-path imports only (so neither pulls `node:fs` into a browser bundle nor `globalThis.indexedDB` into Node):

```
import { FilePersistence }      from '@axona/protocol/persistence/file.js';      // Node
import { IndexedDBPersistence } from '@axona/protocol/persistence/indexeddb.js'; // browser

const peer = new AxonaPeer({ nodeIdentity, domain, node, transport,
  persist: new FilePersistence({ dir: './state' }) });
```

Transport + protocol surface

> **You probably don't need this part.** Everything an application calls is in the Application surface (§§1–9). These sections are for people implementing transports, embedding the kernel, or building tooling.

10. Contracts

Abstract base classes that transports / DHT implementations extend. Application code rarely uses these directly.

`Transport` (abstract class)

```
class MyTransport extends Transport {
  async start(localNodeId)         {}
  async stop()                     {}
  async send(peerId, type, body)   {}   // request/response
  async notify(peerId, type, body) {}   // fire-and-forget
  async openConnection(peerId)     {}
  async closeConnection(peerId)    {}
  isConnected(peerId)              {}
  getLatency(peerId)               {}   // RTT ms or -1
  onRequest(type, handler)         {}
```

```
  onNotification(type, handler)    {}
  onPeerDied(handler)              {}
}
```

Full surface in `@axona/protocol/contracts/Transport.js`.

`DHT` (abstract class)

The per-node routing/pub-sub contract `AxonaPeer` implements (`getSelfId`, `findKClosest`, `routeMessage`, `sendDirect`, `onRoutedMessage`, `lookup`, …).

`BootstrapService` (abstract class)

For peer discovery (signaling brokers, seed lists, DNS-SD). Rarely needed by app code.

---

11. Sim transport

In-process router for tests + simulators. Environment-neutral, ships in the main barrel.

`new SimNetwork({ latencyFn? })`

```
import { SimNetwork } from '@axona/protocol';
const net = new SimNetwork();                          // 0 ms edges
const net2 = new SimNetwork({ latencyFn: () => 50 });  // 50 ms each way
```

`simTransport({ network, identity, heartbeatMs?, ... }) → SimTransport`

Factory that builds + wires a `SimTransport` onto a `SimNetwork`.

```
import { SimNetwork, simTransport } from '@axona/protocol';
const network = new SimNetwork();
const t = simTransport({ network, identity: node, heartbeatMs: 0 });
await t.start(node.id);
await t.openConnection(other.id);
```

`heartbeatMs: 0` disables heartbeats (recommended for tests). `SimTransport` is also exported directly for `new SimTransport({...})`.

---

12. Web transport

The production browser transport (a WebSocket to the bridge for bootstrap + signaling, plus a WebRTC mesh). Sub-path import:

```
import { webTransport } from '@axona/protocol/transport/web/index.js';
```

```
const transport = webTransport({
  bridgeUrl:   'wss://testnet.axona.net',  // or wss://bridge.axona.net (production)
  identity:    node,                       // from createNodeIdentity — signs the handshake
  peerVersion: '4.22.0',                   // your app version (gated by the bridge)
  reconnect:   true,
});
await transport.start();                  // resolves after the bridge handshake
```

Beyond the `Transport` contract it exposes an observability surface: `transport.bridgeState`, `bridgeInfo`, `bridgeRtt`, `bridgeNodeId`, `onBridgeState(cb)`, `onWelcome(cb)`, `boundPeers()`, `reconnectNow()`.

Each mesh link's `axona/5` proof is bound to its DTLS certificate fingerprint, so a fingerprint-rewriting bridge cannot transparently MITM "direct" peer traffic — the bridge is trusted for signaling only.

---

13. Handshake

Wire-version handshake helpers used by transports on a fresh signaling channel. Most app code never touches these — they're plumbed into the transport factories.

```
buildClientHello({ version, wireVersion?, capabilities? })          // → client → server
 ↪ frame
buildServerHello({ version, wireVersion?, minPeerVersion, downloadUrl }) // → server → client
 ↪ frame
parseHello(frame)                                                   // → parsed hello
parseVersion(str)                                                   // → { major, minor,
 ↪ patch }
compareVersions(a, b)                                               // → -1 | 0 | 1
wireCompatible(a, b)                                                // → boolean
performClientHandshake(channel, opts)                               // → handshake result
performServerHandshake(channel, opts)
```

The constants `WIRE_VERSION`, `KERNEL_VERSION`, `UPGRADE_CLOSE_CODE` come from the same module — see §23.

---

14. Authenticated-identity handshake (axona/5)

Re-exported so consumers driving their own channel lifecycle (the bridge's embedded peer, custom transports) can build/verify authenticated hellos with the same primitive the web transport uses.

```
buildAuthHello(opts)                // → signed hello proving nodeId ownership
verifyAuthHello(hello, opts)        // → verification result
pubkeyMatchesNodeId(pubkey, nodeId) // → boolean (BIND check)
```

```
makeNonce()                          // → fresh challenge nonce
cbvFromNonces(...)                   // → channel-binding value from nonces
cbvFromFingerprints(...)             // → channel-binding value from DTLS fingerprints
AUTH_PROTO                           // 'axona/5'
```

The proof binds (BIND: pubkey hashes to the id), (POSSESS: Ed25519 signature), and (CHANNEL: bound to this live connection) so a captured proof can't be replayed onto another link.

---

15. Bridge directory

Bridges advertise on a public topic; clients collect them at launch and fail over.

```
BRIDGE_DIRECTORY_TOPIC               // 'axona:bridge-directory'
BRIDGE_ENTRY_MAX_AGE_MS              // 48 h
buildBridgeEntry({ url, lat, lng, label?, ver?, turn?, ts? })  // → directory entry
validateBridgeEntry(msg)             // → boolean (well-formed + fresh)
rankBridges({ ... })                 // → ordered candidate list (reputation + proximity)
haversineKm(a, b)                    // → great-circle km between two { lat, lng }
```

```
import { BRIDGE_DIRECTORY_TOPIC, buildBridgeEntry } from '@axona/protocol';
const entry = buildBridgeEntry({ url: 'wss://bridge.axona.net', lat: 38, lng: -77 });
await peer.pub({ name: BRIDGE_DIRECTORY_TOPIC, region: 'useast' }, entry, { signWith: me });
```

---

16. Low-level pub/sub builders

The signed wire record behind `peer.kill`. Apps use `peer.kill`; this is for protocol implementors and custom roots.

```
buildKill({ topicId, msgId, ts?, seq?, identity })                    // → signed kill
verifyKill(kill)                                                       // → verification
 ↪  result
KILL_DOMAIN     // 'axona:pubsub-kill:v1'
```

> The `unpub` wire record was **removed in v4.3.0** (kill is the single retraction primitive). `touch` is **deprecated** — its handler is a no-op; the `buildTouch`/`verifyTouch` helpers remain only for wire back-compat.

`AxonaManager` (the pub/sub state machine), `AxonaDomain`, `NeuronNode`, `Synapse`, `Subscription`, `DHTNode`, and `GEO_CELL_BITS` (= 8) are also exported for implementors building custom engines.

Low-level utilities

> **Internals.** Pure helpers the kernel itself is built from. Apps import these only for unusual jobs (custom signing, ID math, region tables).

17. Ed25519 helpers

Web Crypto Ed25519 wrappers — useful for signing/verifying outside `buildEnvelope`.

```
const { publicKey, privateKey } = await generateKeyPair({ extractable: true });
const pubHex  = await exportPublicKey(publicKey);   // 64-char hex
const pubKey  = await importPublicKey(pubBytesOrHex);
const sig     = await sign(privateKey, dataBytes);  // Uint8Array(64)
const ok      = await verify(pubKeyOrBytes, dataBytes, sig);

const signer   = makeSigner(privateKey);   // (bytes) => Promise<sig>
const verifier = makeVerifier(pubKey);      // (bytes, sig) => Promise<boolean>
```

Implementation-agnostic: substitute `@noble/ed25519` on runtimes without Web Crypto Ed25519 (Chrome <110, Safari <17, Firefox <130, Node <20).

---

18. Hex / ID math

264-bit identifier math — nodeId and topicId share the same keyspace (`[8-bit S2 prefix] || [256-bit hash]`).

```
ID_BITS                    // 264
HASH_BITS                  // 256
S2_BITS                    // 8
HEX_CHARS                  // 66
MAX_ID                     // BigInt (2**264 - 1)
MAX_HASH                   // BigInt (2**256 - 1)
MAX_S2                     // 255

toHex(bigint)              // → 66-char lowercase hex
fromHex(hex)               // → BigInt
isHexId(s)                 // → boolean (valid 66-char hex id)
assembleId(s2Prefix, hash) // → BigInt — 8-bit prefix || 256-bit hash
extractS2Prefix(idBig)     // → number 0..255 (top byte)
extractHash(idBig)         // → BigInt (bottom 256 bits)
s2PrefixOfHex(hex)         // → number 0..255
xorDistance(a, b)          // → BigInt
stratumOf(a, b)            // → 0..264 — leading-zero count of (a ^ b)
clz264(bigint)             // → 0..264 — leading-zero count
randomU256()               // → BigInt — cryptographically random
```

---

19. S2 geographic cells

The 8-bit S2 cell that occupies the top byte of every id. The cellId matches the top 8 bits of Google S2's level-3 cell ID (full interop).

```
geoCellId(lat, lng, bits)   // → integer — S2 cell ID at the given bit depth
geoCellCenter(cellId)       // → { lat, lng } — cell center
geoCellCorners(cellId)      // → corner coordinates
geoCellFace(cellId)         // → 0..5 — which cube face
geoCellSubCenters(cellId)   // → [center0, center1] — the two level-3 sub-cell centers
geoCellHalf(lat, lng)       // → 0 | 1 — which sub-cell a point falls in
isValidCellId(cellId)       // → boolean (in [0, 192))
S2_FACES                    // 6
S2_CELL_COUNT               // 192
S2_RESERVED_FROM            // 192
```

---

20. Region names

Each of the 192 region codes carries exactly one human-readable name, so a region always presents the same label. Where a cell straddles ocean and land the land name wins; a multi-country cell takes its dominant city.

```
REGION_NAMES                // frozen string[] indexed by code [0,192)
regionName(code)            // → string | null     e.g. regionName(0x89) → 'useast'
regionNames(code)           // → [name] | null      deprecated one-element shim
regionCode(name)            // → number | null      inverse (canonical code for a
 ↪ multi-cell name)
resolveRegion(nameOrCode)   // → number | null      accepts 'useast' | '0x89' | '137' | 137
regionNameForLatLng(lat, lng) // → string           region name for a coordinate
regionCenter(nameOrCode)    // → { lat, lng } | null cell center (default placement for
 ↪ topics)
POPULATED_REGIONS           // frozen [{ code, name }] for every non-open-ocean cell
```

```
import { regionCode, regionCenter, POPULATED_REGIONS } from '@axona/protocol';
regionCode('useast');       // 137
regionCenter('useast');     // { lat, lng } — mint a node identity in a region
POPULATED_REGIONS.length;   // count of real, inhabited cells (for a region picker)
```

Names match `/^[a-z0-9_]{1,8}$/`; open-ocean cells are `<ocean3>_<hex>` (`pac_68`, `atl_0a`, …) and are excluded from `POPULATED_REGIONS`.

---

21. Geo helpers

```
haversine(lat1, lng1, lat2, lng2)        // → km between two points
roundTripLatency(lat1, lng1, lat2, lng2) // → ms — rough fiber estimate
randomU32()                               // → 0..2**32 - 1
```

The remaining `geo.js` helpers (continent detection, XOR routing-table builders, etc.) are reachable via `@axona/protocol/utils/geo.js`.

---

22. Proof-of-work scaffolding

E-1 / Stage 2 transport + publish PoW. **Inert at difficulty 0** — every nonce is `''` and verification passes trivially until difficulty is raised. Apps don't normally touch these; identities carry their PoW nonce automatically.

```
POW_DOMAIN                                  // 'axona:pow:v1'
POW_DIFFICULTY                              // { transport: 0, publish: 0 } (frozen
↪ defaults)
powDifficulty(role)                         // → current difficulty for 'transport' |
↪ 'publish'
setPowDifficulty(role, n)                   // set difficulty (testing / calibration)
resetPowDifficulty()                        // restore defaults
powMint({ pubkeyHex, role?, difficulty?, maxTries? })  // → Promise<nonce string>
powVerify({ pubkeyHex, nonce, role?, difficulty? })    // → Promise<boolean>
powBits({ pubkeyHex, nonce, role? })        // → Promise<number> leading-zero bits of the
↪ puzzle hash
powCalibrate({ ms? })                       // → Promise<calibration> bits/ms estimate
```

The puzzle hashes `H(domain || role || pubkey || nonce)` — difficulty is on a **separate puzzle hash, never on the node address**, so raising it introduces no keyspace skew.

---

23. Constants

```
WIRE_VERSION         // '4.0'      — wire format major.minor (bridges gate on this)
KERNEL_VERSION       // '4.22.0'   — kernel semver (npm release tag)
AUTH_PROTO           // 'axona/5'  — authenticated-identity handshake tag
UPGRADE_CLOSE_CODE   // 4426       — WebSocket close code for a version mismatch
ENVELOPE_DOMAIN      // 'axona:pubsub-envelope:v2'
MAX_PUBLISH_SKEW_MS  // 300000     — freshness window (5 min)
ID_BITS              // 264
HEX_CHARS            // 66
```

`WIRE_VERSION` is what bridges enforce gates against; `KERNEL_VERSION` is informational.

`ANONYMOUS` (sentinel)

The sentinel for an intentionally unsigned publish. Anonymity must be explicit — omitting a signer is an error, never silent anonymity.

```
import { ANONYMOUS } from '@axona/protocol';
await peer.pub({ region: 'useast', name: 'lobby' }, msg, { signWith: ANONYMOUS });
```

Appendix: version history for migrators

(Only relevant if you have code from an earlier kernel line. New readers: skip.)

What changed in v4.17.0–v4.22.0 (2026-07-14 roll; v4.21.0 promoted to production)

No API change at all — every release in this span is behavioral: root lifecycle, departure, and history-convergence work, validated by an overnight production-shaped soak per release.

- **Root election + reconciliation (v4.17.0–v4.19.2).** Faster promotion when a topic's root churns out; wrong root claims converge without flapping (strictly-closer beacon deferral, periodic root self-verification, and guards for unmeshed and cross-region edge cases). One durable root per topic, kept true under churn.
- **Departure-side durability (v4.19.4–v4.19.5).** `peer.leave()` now hands off **every** rooted topic's history inside its time bound (parallel heir resolution + an iterative-lookup fallback for thin-tabled leavers), and the heir can no longer defer its claim back to the node that just left. A burst publisher's topics survive its departure intact.
- **Kernel consolidation (v4.19.6–v4.21.0).** The refactor program: a frozen invariants contract (`INVARIANTS.md`), every root transition through one state machine (`rootClaim.js`), and the pub/sub manager split along its seams. Structural only — verified behavior-preserving.
- **Split-history union (v4.22.0).** After a root transition, the old and new holders converge to the **union** of cache + tombstones (the subscribe now advertises its oldest stamp so a root pulls history that sits *below* its own high-water; roots union-ingest cohort pushes). Closes the last known replay gap: a fresh `since:'all'` subscriber attaching mid-transition received only the post-transition half (~1 in 15 root transitions); soak-validated to full-timeline recovery.

What changed in v4.12.0–v4.16.1 (2026-07-02 testnet roll)

- **`connect()` (v4.16.0)** — the one-call bootstrap above; no other API change.
- **Immediate metrics answer (v4.16.1)** — a topic root publishes the FIRST metric snapshot the moment a metrics lease arms, so it arrives at routing latency (measured ~0.3 s on testnet) instead of on the next 5 s tick. Cadence (~20 s) and lease (~70 s) unchanged.
- **Demand-driven metrics (v4.12.0)** — snapshots publish to `metricTopic(T)` only while a subscriber's lease is fresh; `peer.metrics()` unchanged. The relay's old push-metrics loop is retired. See the timing contract in §4.3.

- **Region-occupancy rule (v4.13.0, gated OFF in v4.15.0)** — the kernel can enforce region-homogeneous topic service (`configureRegionLock({ enforce })`); disabled by default until regional coverage justifies it.
- **BigInt id invariant (v4.14.0)** — internal ids validated at one gate (`asId`); `NeuronNode`/`Synapse` now accept hex ids directly. `asId` exported.

What changed in the 4.x line (since v3.6.0)

The **public API surface is unchanged** from v3.6.0 — same identity factories, topic descriptors, and `pub`/`sub`/`pull`/`kill`/`host` signatures. The 4.x work is a routing-and-reliability rewrite *under* that surface, so existing code keeps working while delivery gets more robust:

- **Routing-only axonic-tree pub/sub (v3.14, clean break).** Topics route to an emergent root (the live node XOR-closest to the topic id) with no separate overlay to maintain. Behavioral only — no call changed.
- **K-closest cohort distribution (v4.10.0).** A topic's authoritative state (messages *and* retractions) is replicated across the closest-K nodes, not one root. A `kill` reaches the whole cohort and internal transfers carry their tombstones, so a killed message can't resurface to a late subscriber, and a publish survives the root churning out.
- **Metrics rebuilt (v4.10.1).** `rootedTopics()`/`peer.metrics()` were silently dead across early 4.x; rebuilt with two new fields — **`seq`** (dense message counter) and **`cohortSize`** — and cohort-aware aggregation (see §4.3).
- **Cold-publish burst (v4.11.0).** A freshly-joined node's first publishes are automatically re-sent a few times over the first second, so a cold-start publish isn't lost while the routing table warms. No app action — `pub` is unchanged.
- **Nearest-replica reads (v4.11.1+).** `pull` is answered by the first replica the request reaches instead of always the root: exact for `pull(msgId)`, and *recent* (eventually-consistent) for pull-latest, which spreads a hot read path off the root (see §4.3).

What changed since v2.x

The pub/sub and identity surfaces were redesigned in the v3 line. If you are migrating, the load-bearing differences are:

- **Two identity factories.** `createNodeIdentity({ lat, lng })` mints the *connection* key (the nodeId); `createAuthorIdentity()` mints a location-free *authorship* key (the Author ID). The old single `deriveIdentity()` is gone.
- **Topics are descriptors, not strings.** A topic is `{ region?, owner?, name, write? }`. `peer.pub` takes the descriptor; `peer.sub` / `pull` / `metrics` take the descriptor **or** a 66-hex topic ID. There is no `publisher`/`publishId` argument anywhere.
- **Signing is explicit per publish.** `peer.pub(topic, message, { signWith })` requires a signer — an author identity or the `ANONYMOUS` sentinel. There is no default author, the node key never signs publishes, and there is no `sign: false`.
- **The envelope carries the topic descriptor.** A signed publish binds the exact `{ region, owner, name, write }`, so a storing node recomputes the topic ID and enforces the write policy (`signer === owner` for owned topics).

A word on stability

`@axona/protocol` 4.x is stable for the application surface (§§1–9). The transport + protocol surface (§§10–16) is stable but expect occasional additions. Low-level utilities (§§17–23) may grow but won't change incompatibly.

For changelog and migration notes, see https://github.com/axona-net/axona-protocol/releases.